

IPL Match Strategy & Performance Analysis

Python | Pandas, NumPy, SciPy, Matplotlib, Seaborn • Dataset: 636 IPL matches & 150,460 ball-by-ball deliveries (2008–2017)

Project Overview

This project analyzes a decade of Indian Premier League (IPL) cricket data to evaluate toss strategy, scoring patterns across different phases of a match, and player performance. The goal was to move beyond surface-level statistics and validate findings using formal hypothesis testing, in line with standard data analytics practice.

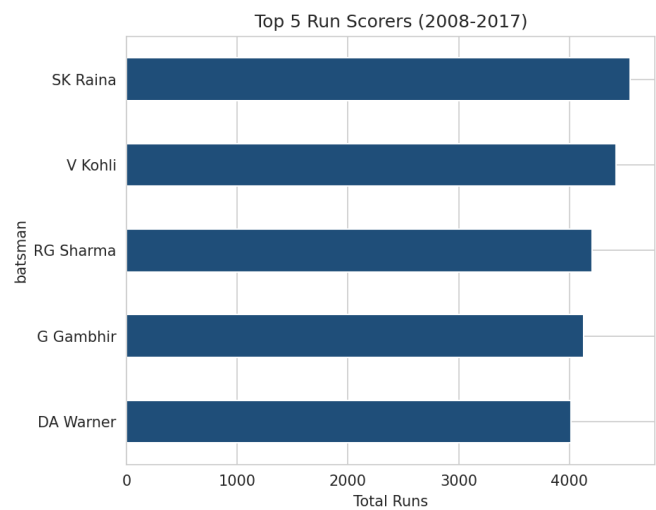
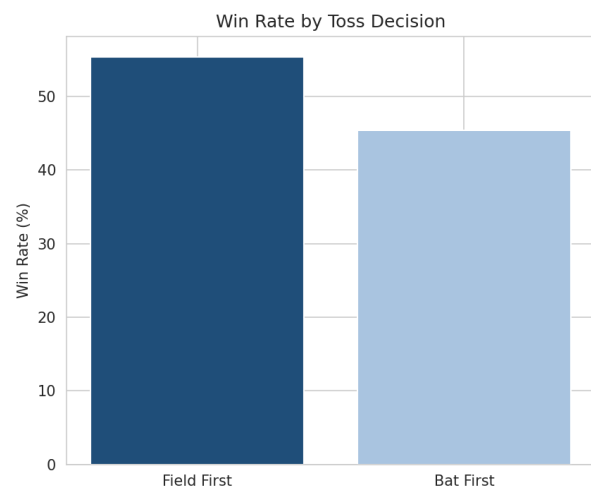
Methodology

- Loaded and merged two relational datasets – match-level outcomes (matches.csv) and ball-by-ball delivery records (deliveries.csv) – using Pandas.
- Computed toss-decision win rates and tested statistical significance using a Chi-Square Test of Independence.
- Segmented deliveries by over number to compare powerplay (overs 1–6) and death-over (overs 16–20) scoring.
- Aggregated batting and bowling statistics across all ten seasons to surface top performers and the most economical death-over bowlers (minimum 200 balls bowled, to ensure sample reliability).
- Visualized key comparisons using Matplotlib and Seaborn.

Key Findings

- **Toss strategy matters:** teams that won the toss and chose to field first won 55.4% of matches (n=363), versus 45.4% when choosing to bat first (n=273). A Chi-Square test confirmed this difference is statistically significant (Chi-Square = 5.78, p = 0.016), not random variation.
- **Chasing is the stronger position:** consistent with the above, 339 matches were won while chasing (by wickets) compared to 287 won while defending a total (by runs).
- **Powerplay outproduces the death overs:** innings averaged 90.3 runs in the powerplay (overs 1–6) versus 83.6 runs in the death overs (overs 16–20) across this 2008–2017 window.
- **Top run scorers (2008–2017):** SK Raina (4,548), V Kohli (4,423), RG Sharma (4,207), G Gambhir (4,132), and DA Warner (4,014).
- **Most economical death-over bowlers** (min. 200 balls bowled in overs 16–20): SP Narine (7.14), DE Bollinger (7.36), SL Malinga (7.43), MA Starc (7.57), and R Ashwin (7.58 runs/over).

Visualization



Left: win rate by toss decision (field vs. bat first). Right: top 5 run scorers, 2008–2017.

Appendix – Analysis Script

Full Python script used to produce the findings and chart above.

```
"""
IPL Match Strategy & Performance Analysis
-----
Dataset : IPL ball-by-ball + match data, 2008-2017 (636 matches, 150,460 deliveries)
Source  : Public IPL dataset (originally published on Kaggle by Manas Garg)
Goal    : Evaluate toss strategy, scoring patterns across match phases, and
          top player performance to surface data-backed game strategy insights.

Libraries: Pandas, NumPy, SciPy, Matplotlib, Seaborn
"""

import pandas as pd
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")

# -----
# 1. Load data
# -----
matches = pd.read_csv("matches.csv")
deliveries = pd.read_csv("deliveries.csv")

print(f"Matches: {matches.shape[0]} | Deliveries: {deliveries.shape[0]}")
print(f"Seasons covered: {sorted(matches['season'].unique())}")

# -----
# 2. Toss strategy vs match outcome
# -----
toss_match_win_rate = (matches["toss_winner"] == matches["winner"]).mean()
print(f"\nOverall toss-winner win rate: {toss_match_win_rate * 100:.1f}%")

field_decision = matches[matches["toss_decision"] == "field"]
bat_decision = matches[matches["toss_decision"] == "bat"]

field_win_rate = (field_decision["toss_winner"] == field_decision["winner"]).mean()
bat_win_rate = (bat_decision["toss_winner"] == bat_decision["winner"]).mean()

print(f"Win rate after choosing to FIELD: {field_win_rate * 100:.1f}% (n={len(field_decision)})")
print(f"Win rate after choosing to BAT: {bat_win_rate * 100:.1f}% (n={len(bat_decision)})")

# Statistical significance check: is toss decision independent of winning?
contingency = pd.crosstab(matches["toss_decision"], matches["toss_winner"] == matches["winner"])
chi2, p_value, dof, expected = stats.chi2_contingency(contingency)
print(f"Chi-Square test -> chi2={chi2:.3f}, p-value={p_value:.4f}")

# -----
# 3. Scoring patterns: Powerplay (overs 1-6) vs Death overs (16-20)
# -----
powerplay = deliveries[deliveries["over"] <= 6]
death_overs = deliveries[deliveries["over"] >= 16]

pp_avg = powerplay.groupby("match_id")["total_runs"].sum().mean()
death_avg = death_overs.groupby("match_id")["total_runs"].sum().mean()

print(f"\nAvg powerplay runs/innings: {pp_avg:.1f}")
print(f"Avg death-over runs/innings: {death_avg:.1f}")

# -----
# 4. Top performers
# -----
top_batsmen = (
    deliveries.groupby("batsman")["batsman_runs"].sum().sort_values(ascending=False).head(5)
)
print("\nTop 5 run scorers:\n", top_batsmen)
```

```

death_bowling = death_overs.groupby("bowler").agg(runs=("total_runs", "sum"), balls=("ball", "count"))
death_bowling = death_bowling[death_bowling["balls"] >= 200]
death_bowling["economy"] = death_bowling["runs"] / (death_bowling["balls"] / 6)
top_death_bowlers = death_bowling.sort_values("economy").head(5)
print("\nBest death-over economy (min 200 balls bowled):\n", top_death_bowlers)

# -----
# 5. Win type split (chasing vs defending)
# -----
win_by_runs_count = (matches["win_by_runs"] > 0).sum()
win_by_wickets_count = (matches["win_by_wickets"] > 0).sum()
print(f"\nMatches won defending (by runs): {win_by_runs_count}")
print(f"Matches won chasing (by wickets): {win_by_wickets_count}")

# -----
# 6. Visualizations
# -----
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].bar(["Field First", "Bat First"], [field_win_rate * 100, bat_win_rate * 100],
            color=["#1F4E79", "#A9C4E0"])
axes[0].set_title("Win Rate by Toss Decision")
axes[0].set_ylabel("Win Rate (%)")

top_batsmen.plot(kind="barh", ax=axes[1], color="#1F4E79")
axes[1].set_title("Top 5 Run Scorers (2008-2017)")
axes[1].set_xlabel("Total Runs")
axes[1].invert_yaxis()

plt.tight_layout()
plt.savefig("ipl_analysis_charts.png", dpi=150)
print("\nCharts saved to ipl_analysis_charts.png")

```